Technical Document

Service booking systems: Keyloop and its dealerships, and the design of the Unified Service Scheduler

Version	2.0 (English edition)
Date	25 October 2026
Based on	The shared Vietnamese working document "Khảo sát hệ thống đặt lịch bảo dưỡng – Keyloop & đại lý" (revision 14)
Changes from the Vietnamese edition	Translated into English, and updated to the delivered build: the backend now runs on Scala 3 + Play 3, so the sections on the first code version, concurrency and the SOLID review describe the Scala code

1. Summary

Two UK dealerships that use Keyloop (Wilsons Epsom and Busseys) both take service bookings through the same multi-tenant SaaS system at `<dealership>.servicestorefront.co.uk`, most likely Keyloop Service Booking. Their home websites were built by two different agencies, so the similar look of the booking pages is an industry convention rather than shared code.

Keyloop's public API schedules work by **service advisor**. Scenario A asks for something deeper: check that a **service bay** and a **technician with the right skill** are both free. That is where this solution goes further than the existing product.

The delivered solution is a working system: a Scala 3 / Play 3 REST API on PostgreSQL, where three database constraints make double booking impossible, and a React web app for five roles. It is covered by 95 automated tests and a 1000-request load test.

2. Agreed decisions

Scenario A is built with the backend in Scala + Play 3 + PostgreSQL. The scheduling core stays in one module so that double booking is prevented inside one transaction.

Topic	Decision	Reason	Outcome in the build
Scenario	Scenario A, Unified Service Scheduler	Real experience of booking motorbike maintenance in Vietnam	Built
Fully implemented layer	Backend; client mocked with the OpenAPI spec and cURL	The backend is the strongest skill	Built. A working React app was added on top, so the whole journey can be demonstrated

Topic	Decision	Reason	Outcome in the build
Language	Scala (Java as fallback)	Few candidates use it, strong type safety; Keyloop does not mandate one stack	Scala 3.3.6 LTS (also builds and passes on 3.9.0 and 3.10.0)
Framework	Play Framework 3 (on Pekko)	Already familiar; maintained (3.0.11 released May 2026)	Play 3.0.12 with compile-time dependency injection
Database	PostgreSQL + Flyway for migrations	Needs exclusion constraints and strong transactions	PostgreSQL 17, Flyway 11
Double-booking protection	`tstzrange` exclusion constraints + `btree_gist` on bay and on technician; test with 50 parallel requests, only the free capacity succeeds	Enforced in the database, correct even with concurrent requests	Three constraints: bay, technician **and vehicle**. Proved by the 50-request test and the 1000-request load test
Database access	Slick or Doobie (open)	Not yet chosen	**Anorm** with HikariCP: plain SQL with typed parameters, because the constraints and range queries are PostgreSQL-specific anyway
API contract	Tapir on Play, generating OpenAPI and Swagger UI (open)	Meets the API-contract requirement without hand-writing	**Hand-written OpenAPI 3.1** (`conf/openapi.yaml`) served with Swagger UI at `/docs`; a test fails if a route is missing from the contract
Architecture	Hexagonal per module (light "Hive"): Scheduling, Catalog, Customer & Vehicle	Boundaries follow data-consistency boundaries; the compiler checks wrong imports	Seven modules (scheduling, consultation, vehicles, catalog, identity, admin, system) in one sbt project; `ArchitectureSpec` enforces the dependency rule
Scheduling boundary	Appointment, ServiceBay, Technician, Skill in one module	The no-overlap invariant needs one transaction; splitting would need a saga	As decided
Multi-dealership	Every row carries `dealershipId`	Matches the multi-tenant model of servicestorefront	As decided; tests check that staff of one dealership cannot see or change another
Extra features	Alternative slots when full; idempotency key for `POST /appointments`; optional HOLD → CONFIRM	Product thinking; ready for AI agents to call	Alternatives and idempotency key built. HOLD → CONFIRM not built (future work)
Chatbot / LLM	Not built; recorded as future work, linked to Keyloop's AIME	Outside the brief, dilutes the core	As decided
Batch	No batch framework; expired HOLDs filtered in the query; Pekko Scheduler for any small job	The brief has no real batch processing	No batch jobs needed

Topic	Decision	Reason	Outcome in the build
Testing	ScalaTest + ScalaCheck (property-based, no-overlap invariant) + Testcontainers	Verifies AI-written code better than a few unit tests	ScalaTest + ScalaCheck; **embedded PostgreSQL (Zonky)** instead of Testcontainers, so tests need no Docker but still run on the real engine
Observability	Structured logs with correlation ID; OpenTelemetry; metric `booking_conflicts_total`	Actually implemented, not just listed	JSON logs with request ID, Prometheus metrics including `booking_conflicts_total`, audit trail. OpenTelemetry tracing is future work
Sample data	Seed makes and models from NHTSA vPIC; optional VIN check with timeout and fallback	Free API, no key; booking must not depend hard on an outside service	Own Vietnamese seed data; VIN format checked locally. vPIC not used
Working with AI	Write the spec and invariants first; log in `docs/ai-log.md` every AI mistake and its fix; small commits	AI Engineering & Verification carries the most weight	12 entries in `docs/ai-log.md`

3. Product scope

The Priority column was a proposal to meet the Tuesday 27 October deadline. Everything stays in the design documents; the column only says whether it has code.

3.1 User roles

Role	What they do	Source	Priority	Built
System manager	Runs the whole platform, creates and manages dealerships	Original	Future work	**Yes** (onboard and monitor dealerships)
Admin (dealership)	Configures bays, technicians, skills, service packages and opening hours of one dealership	Original	Submission	Yes (bays, technicians, staff; packages and hours are seeded)
Sales	Advises on customisation, sells parts and extra services	Original	Future work	Covered by the service advisor role
Technician	Sees assigned jobs, updates job status	Original	If time allows	**Yes**
Customer (added)	Registers vehicles, books, views and cancels, sends advice requests	servicestorefront booking flow	Submission	Yes
Service advisor (added)	Receives customers, advises before booking, confirms bookings	Keyloop API is built around advisors	Submission	Yes

Role	What they do	Source	Priority	Built
Workshop controller (added)	Sees workshop load, moves a job to another bay or technician	Tribe AI case study: workshop administrators schedule technicians	Future work	No
Parts staff (added)	Handles parts orders	Keyloop Parts Catalogue API has a Parts Advisor role	Future work	No

Simplification adopted: Sales is merged into Service advisor, because both advise before booking.

3.2 Features

Feature	Description	Flow	Priority	Built
Sign in, sign out, permissions	Each role can only call its own APIs; data separated by `dealershipId`	With JWT, sign-out needs short-lived tokens or a revocation list	Submission	Yes, with opaque bearer tokens stored as SHA-256 hashes in `sessions`, so sign-out really revokes
Maintenance packages (main feature)	Customers who know little about vehicles pick a ready-made package with a checklist, duration, required skill and bay type	Vehicle → package → dealership → slot → confirm	Submission	Yes
Inspection on request, with advice	The customer names what to check; an advisor assesses first, then the booking is made	Request → advisor sets duration and skill → customer picks a slot	If time allows	**Yes**
Customisation with advice (secondary)	Same advice flow as inspection on request	As above, plus a quote	Future work	**Yes**
Replacement parts ordering	Order parts, optionally attached to an appointment	A separate domain (stock, price, orders); Keyloop has its own Order Parts API	Future work	No

Inspection on request and customisation share one mechanism, "advice request → appointment". Building it once served both.

4. Implementation

4.1 History

The first version (24 October) was written in TypeScript (Node + Fastify) because the first build environment could not download Scala or Play libraries (Maven Central was blocked). The schema, the hexagonal structure and the double-booking constraints were designed to be portable. On 25 October, with Maven Central reachable, the backend was ported to **Scala 3 + Play 3** with the same modules,

class names and ports, and every test was ported case by case with the same expected values. The TypeScript backend was then removed.

4.2 What is built

- **Roles:** sign-in and sign-out for five roles (Customer, Service advisor, Technician, Dealership admin, System manager).
- **Customer:** maintenance packages, inspection on request and customisation through the advice flow, appointments, cancellation, vehicles, add to calendar.
- **Staff:** the advisor's workshop-day board, the technician's job list, management of bays, technicians and staff, dealership onboarding.
- **Double-booking protection:** three exclusion constraints (bay, technician, vehicle); retry after a lost race; idempotency key; three nearest alternatives when a slot has just been taken.
- **Logging:** JSON logs with a request ID that flows from the browser to the server, Prometheus metrics, an audit trail of appointment status changes.
- **Not built:** parts ordering, rescheduling (today: cancel and rebook), SMS / Zalo notifications, OpenTelemetry tracing, HOLD → CONFIRM.
- **AI log:** `docs/ai-log.md` records 12 real defects caught in AI-written code, for the AI Collaboration Narrative.

4.3 Stack

Layer	Technology
Language and runtime	Scala 3.3.6 (LTS), JDK 21 (virtual threads)
Web framework	Play Framework 3.0.12 on Pekko HTTP, compile-time DI (`AppLoader`)
Database	PostgreSQL 17, Flyway 11.20 migrations, Anorm 2.7, HikariCP 6.3
Embedded database	Zonky embedded-postgres (PostgreSQL 17.6 binaries) for `sbt run` and tests
JSON	play-json, Jackson pinned to 2.19.1
Logging and metrics	Logback with logstash-logback-encoder, Prometheus Java client 1.10
API documentation	OpenAPI 3.1, Swagger UI 5.17
Tests	ScalaTest, ScalaCheck
Frontend	React 19, React Router 7, TanStack Query 5, Vite 8, TypeScript 6, Luxon

4.4 How a booking stays correct

1. `GET /api/availability` computes, for each start time on the dealership's grid, how many (bay, technician) pairs are free for the whole job: a bay of the required type, and a technician with the required skill who is on shift. This is pure code in `domain/Availability.scala`.
2. `POST /api/appointments` checks again inside a transaction, picks a pair and inserts. **Best fit:** the technician with the fewest skills is chosen first, so specialists stay free for work only they can do; ties go to whoever has the least booked that day.
3. PostgreSQL is the final judge. Three `EXCLUDE USING gist` constraints on `tstzrange` periods make overlapping bookings of the same bay, technician or vehicle impossible. Ranges are half-open, so 09:00–10:00 and 10:00–11:00 do not clash.

4. If a concurrent request won the same pair (SQLSTATE `23P01`), the service retries with fresh data, up to five times. If nothing is left, it answers `409 SLOT_UNAVAILABLE` with the nearest alternatives.
5. An `Idempotency-Key` header makes retries and double taps safe: the same key returns the original appointment (`200`, `Idempotent-Replayed: true`). An advisory lock per customer and key queues duplicate requests instead of letting them race.

The data model is drawn in `diagrams/erd.png`: business tables in green, authentication and authorisation tables in red, framework tables in orange.

5. Concurrency and load

The system answers all 1000 requests fired at the same instant, with no errors and no double booking. It uses **Java 21 virtual threads**.

5.1 Concurrency model

- **Non-blocking HTTP, blocking code on virtual threads:** Play's HTTP threads never block. Each use case runs on its own virtual thread (`Blocking` in `http/Http.scala`). JDBC calls and password hashing block that virtual thread, which costs a few hundred bytes, not an operating system thread, so 1000 waiting requests cost memory, not 1000 threads.
- **Real parallelism is in the database:** each API instance has a pool of 20 PostgreSQL connections (HikariCP). Nested calls on one thread reuse the same connection, so a burst cannot starve the pool.
- **Correctness does not depend on threads:** the no-double-booking rule is enforced by three PostgreSQL `EXCLUDE` constraints, so it holds for any number of instances.
- **Scaling out:** run more API instances behind a load balancer. The API keeps no state in memory; sessions are in the database.

5.2 Load test results

`sbt loadTest` (or `npm run test:load`) starts the API on a real socket and fires 1000 requests at once over HTTP, three times. Measured on 25 October 2026 on a 2-vCPU machine, with the API, PostgreSQL and the load generator sharing the two cores.

Burst of 1000 requests	Result	Wall time	p50 / p99
Same slot, 4 free bay + technician pairs	exactly 4 × 201, 996 × 409 with alternatives, 0 errors	5.5 s	4.4 s / 5.3 s
Spread over 54 slots, 20 pairs	1000 × 201, 0 overlaps in the database	3.2 s	2.2 s / 3.0 s
Availability reads	1000 × 200	2.1 s	1.6 s / 2.0 s

Latency here is queueing on two shared cores, not a limit of the design.

5.3 Problems found by load testing (fixed)

Problem	Cause	Fix	Before → after
Pressing Book repeatedly with one Idempotency-Key: in about 1 run in 4, some requests got 500	PostgreSQL broke a deadlock (`40P01`) between the unique-key check and the exclusion constraint	Advisory lock per customer + key so the requests queue; deadlocks are still retried as a safety net; regression test runs 10 rounds and fails without the fix	test: 14 s → 0.4 s
1000 availability reads took 43 s (first version)	About 40% of CPU was time-zone arithmetic per technician per slot	Compute each slot's time window once, use a fixed offset on days without a clock change, compute booked minutes outside the sort	43 s → 4.9 s
Scala port: requests hung for 30 s and failed under a burst	Connection-pool starvation: a use case held a connection while a nested port call waited for a second one; all 20 were taken	A thread-local session in `PgDb`: nested `run` / `tx` calls reuse the current connection or join the open transaction	1000 requests answered
Scala port: the first burst failed with "received no bytes"	Pekko HTTP's accept backlog (default 100) overflowed when 1000 clients connected at once	`pekko.http.server.backlog = 1024`	0 dropped connections

For the interview: "Play keeps the HTTP threads non-blocking and runs each use case on a Java 21 virtual thread, so blocking JDBC code stays simple and 1000 waiting requests cost memory, not threads. Correctness under contention lives in database constraints, and a 1000-request load test proves it." Details: the Load test section of the README and entries 6, 7, 11 and 12 in `docs/ai-log.md`.

6. SOLID review (25 October 2026)

The code meets SOLID after one refactoring pass: the review found eight violations and fixed all eight. The API, the tests and the 1000-request load test gave the same results as before. The dependency rule is checked by an automated test, not only by review. The review was done on the first version; the Scala backend was written with the same structure, so every fix holds there.

#	Principle	Before	After (Scala names)
1	SRP	`BookingService` (403 lines) did availability, booking, read views and status changes	Split into `AvailabilityService`, `BookingService`, `AppointmentService`; shared rules in `WorkshopContext`
2	SRP	Admin, vehicles and system routes ran SQL inside the handler	Each module has a controller (HTTP), `application/` (use cases), `adapters/Pg*.scala` (SQL), `Ports.scala` (traits)
3	DIP	`BookingService` created a concrete catalogue repository	It receives a `CatalogReader` trait; only `AppComponents` picks implementations

#	Principle	Before	After (Scala names)
4	DIP	Booking and consultation called a vehicle lookup in the vehicles HTTP adapter	Both depend on the `OwnedVehicles` trait
5	DIP	`BookingService` read PostgreSQL error codes (`23P01`, `40P01`)	Adapters translate them into `SlotTakenError`, `VehicleBusyError`, `TransientStorageError`; SQL codes exist only in the adapter layer
6	DIP	The database port exposed driver types	`Db` and `DbSession` are our own traits; the application layer never imports JDBC or Anorm
7	ISP	Booking depended on the whole `ConsultationService`, the guard on the whole `IdentityService`, every use case on an 11-method store	`ConsultationBookingPort` (3 methods), `Authenticator` (1 method), the store split into `ResourceCalendar`, `AppointmentReader`, `AppointmentWriter`
8	OCP	Status-transition errors mapped by a chain of `if`s	A `TransitionErrors` table: a new rule is one new entry

6.1 Evidence

- **LSP:** `BookingService` runs on in-memory fakes (`BookingServiceFakesSpec`, no database) and on PostgreSQL (integration tests) with the same expectations.
- **DIP is enforced:** `ArchitectureSpec` fails if a domain, application or port file imports an adapter, the HTTP layer, `play.api.mvc`, Anorm, JDBC, the driver or the SQL helpers. A deliberate violation was tried to confirm the test catches it.
- **Checked:** `-Wunused:all -Werror` compiles clean; 95 tests pass; the 1000-request load test passes; the packaged server boots and the web app runs every page in a headless browser with no errors.

6.2 Bugs found during the review (fixed)

Bug	Fix
The slot grid ignored `maxDaysAhead`: a far date looked free, and a suggested alternative could be outside what the dealership accepts, so booking it failed with `TOO_FAR_AHEAD`	One shared `lastBookableDay` rule; a property test checks that every slot shown as free can be booked (it fails without the fix)
A job the dealership cannot do answered `409` with an empty list	Now `422 NOT_OFFERED_HERE`, documented in the OpenAPI contract
A deadlock outside the booking flow would have answered `500`	Now `503 TRY_AGAIN` with `Retry-After`

6.3 Kept on purpose

Metrics and the logger are shared across the system; request shapes (play-json `Reads`) sit next to the use cases; ports take a `DbSession` so one use case can write several tables in one transaction; password hashing is a plain object; seed, migrations and `/health` run SQL directly because they are infrastructure tools. Details in `docs/solid-review.md`.

7. Websites surveyed

Both dealerships point their "Book Service" button at the same platform, with a different subdomain each.

Site	Role	Technology observed	Note
wilsons.co.uk	Wilsons Epsom (Surrey) home site	NetDirector, footer links to keyloop.com; images from images.netdirector.co.uk; static files on AWS CloudFront	"Service & MOT" leads to the booking site
wilsons.servicestorefront.co.uk	Wilsons booking	Could not be opened with the HTML reading tool	According to Keyloop, Wilsons takes about 500 bookings a month through Service Booking
busseys.co.uk	Busseys (Norfolk) home site	WordPress, built by the agency Netmatters	Not a Keyloop platform
busseys.servicestorefront.co.uk	Busseys booking	Could not be opened with the HTML reading tool	Busseys is a Service Hub pilot site

The `<dealership>.servicestorefront.co.uk` pattern shows one system shared by many dealerships, one subdomain each. It is not 100% confirmed to be a Keyloop product, because the pages could not be opened from the research environment.

8. Keyloop's Service Appointment Booking API

The documentation is public on Keyloop Developer, but calling the API requires joining the Partner Programme and paying per call. It runs on the Autoline, Aswin, Automaster, Autoline Drive and WinDrakkar DMSs.

Method	Endpoint	Purpose
GET	`/appointments/service-categories`	Service categories
GET	`/appointments/service-products`	Specific services
GET	`/appointments/options`	Booking options (for example a courtesy car)
GET	`/advisors`, `/advisors/{advisorId}`	Service advisors
GET	`/advisors/availability`	Advisors' free time
GET	`/advisor-groups`	Advisor groups
POST	`/appointments/availability`	Find free slots
POST	`/appointments/options/availability`	Free slots for an option
POST	`/appointments/service-products/availability`	Free slots for a service
POST	`/appointments`	Create an appointment
DELETE	`/appointments/{appointmentId}`	Cancel an appointment

Notable rules:

- Free slots are checked in real time against the workshop diary in the DMS.
- An appointment can only be cancelled while the dealership has not started work on the vehicle.
- No endpoint deals with bays or technician skills.

9. Related products

Product	Owner	What it does	Relevance to Scenario A
Service Hub	Keyloop	Cloud after-sales platform: online booking, check-in, booking management, daily dispatch, workshop management	The closest real product to Scenario A, in the Ownership domain
Tribe AI scheduling engine	Keyloop + Tribe AI	Builds a near-optimal technician plan every morning by job type, skill and bay	Shows Keyloop treats bay and technician scheduling as a real problem
AIME	Motortech.ai, acquired by Keyloop	AI agent that talks to customers and books appointments	Mainly sales; not in scope
Autoflows	Third party	AI voice agent takes slots from Keyloop and writes confirmed bookings back	An example of an AI agent calling a booking API
TruVideo in Service Hub	Keyloop + TruVideo	Vehicle inspection videos attached to the service record	Out of scope
UVeye	UVeye	Automatic vehicle scan in the service lane	Out of scope; context for the video

10. Improvement opportunities for Scenario A

These points come from public documentation and were not verified on the live interface.

Observed today	Improvement proposed	Built
Free slots are computed per advisor	Check at the same time that a bay and a technician with the right skill are free for the whole job	Yes
Double-booking protection depends on the DMS	Enforce it in the database with PostgreSQL exclusion constraints, proved by a concurrency test	Yes
When a slot is full, the customer searches for another time alone	Return the few nearest free slots when the requested time is taken	Yes
No visible hold while the customer fills in the form	A temporary hold with an expiry (HOLD → CONFIRM)	No (future work)
No visible protection against pressing Book twice	Idempotency key for `POST /appointments`, useful also for AI agents such as AIME or Autoflows	Yes

Observed today	Improvement proposed	Built
Cancellation only before work starts	Keep the rule and model the appointment states explicitly	Yes (state machine in `domain/Lifecycle.scala`)
Multi-tenant through subdomains	Tie every row to `dealershipId` from the start	Yes

11. Open checks to do by hand

☐ Open both servicestorefront sites, walk the booking flow up to the confirmation step, and take a screenshot of each step.
☐ Open F12 → Network to see which domain the booking API calls go to.
☐ Look for a "Powered by" line or the Keyloop name on the booking pages.
☐ Watch the Service Hub webinar on keyloop.com to see the workshop-side interface.

12. Sources

- Wilsons Epsom
- Busseys
- Keyloop – Wilsons Epsom testimonial
- Keyloop – Service Hub launch
- Keyloop Developer – Products
- Keyloop Developer – Service Appointment Booking
- Keyloop – Product-specific terms (API plans)
- Tribe AI – Keyloop case study
- IT Brief UK – Keyloop buys Motortech.ai
- Autoflows – Keyloop integration